

UNIVERSIDADE FEDERAL DE SANTA MARIA
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE PROCESSAMENTO DE ENERGIA ELÉTRICA



Polimorfismo

Curso de Engenharia de Controle e Automação
DPEE1090 - Programação orientada a objetos para automação

Prof. Renan Duarte

1º semestre de 2024

Sumário

Polimorfismo

- Definição de polimorfismo e sua importância na POO
- Polimorfismo estático: sobrecarga de métodos e operadores
- Polimorfismo dinâmico: substituição de métodos em tempo de execução

Introdução

Relembrando – Assinaturas de métodos

Refere-se ao conjunto de informações que identificam de forma única um método em uma classe ou interface. Inclui o nome do método e a lista de parâmetros (tipo e ordem).

A assinatura de método **não inclui** o tipo de retorno, os modificadores de acesso ou exceções lançadas pelo método.

Exemplos de métodos com assinaturas diferentes:

```
public void setNome(String nome) { ... }  
public void setNome(String nome, int idade) { ... }  
public void setNome(String nome, int idade, double peso) { ... }  
public void setNome(String nome, int idade, double peso, boolean ativo) { ... }
```

Introdução

Polimorfismo

Poli → Muitas + Morfo → Formas

Em POO, polimorfismo permite que um mesmo nome represente vários comportamentos diferentes.

Exemplo conceitual:

Imagine um controle remoto universal que pode operar diferentes dispositivos eletrônicos como uma TV, um DVD player e um sistema de som. Cada dispositivo possui a sua própria implementação das funcionalidades básicas como ligar/desligar e ajustar o volume, mas todos eles podem ser controlados usando os mesmos botões no controle remoto universal.

Introdução

Tipos de polimorfismo

Existem alguns tipos diferentes de polimorfismo dependendo do contexto em que a mudança de comportamento ocorre. Neste curso, estudaremos os dois tipos principais, classificados de acordo com o momento em que a mudança de comportamento ocorre:

Polimorfismo de sobrecarga (estático ou *Compile-time Polymorphism*) → A resolução do método a ser chamado é feita durante a compilação do programa

Polimorfismo de sobreposição/sobreescrita (dinâmico ou *Runtime Polymorphism*) → A resolução do método a ser chamado é feita em tempo de execução

Polimorfismo de sobrecarga

Ocorre em tempo de compilação

Definição: Múltiplos métodos na mesma classe compartilham o mesmo nome, mas têm diferentes assinaturas

Benefício: Permite o uso de um único nome de método para diferentes operações baseadas nos parâmetros fornecidos

Não envolve herança

Exemplo:

`class Cálculo`

```
    def soma(a, b):  
        return a + b  
    def soma(a, b, c):  
        return a + b + c  
    def soma(a, b, c, d):  
        return a + b + c + d
```

Exemplo 1

Implementar uma classe de um sistema de monitoramento de sensores que possui um método chamado `LerDados` que opera com base nos seguintes critérios:

- Lê dados de um sensor de temperatura, recebendo como parâmetro um identificador do sensor (inteiro).
- Lê dados de um sensor de pressão, recebendo como parâmetro um identificador do sensor (inteiro) e a unidade de medida (string).
- Lê dados de um sensor de umidade, recebendo como parâmetro um identificador do sensor (inteiro), a unidade de medida (string) e a precisão desejada (double).

Polimorfismo de sobreposição

Ocorre em tempo de execução

Definição: Uma subclasse fornece uma implementação específica de um método que já é definido em sua superclasse

Benefício: Permite que a subclasse adapte ou estenda o comportamento de métodos da superclasse

Permite tratar de forma homogênea objetos heterogêneos

Exemplo:

çlắşş Anîñắl

wîşţuắl şţşîng şộn

sêţşşş Ô ắñîñắl gắc ụn şộn

çlắşş Cắçhộssộ Anîñắl

ộwêssîđê şţşîng şộn

sêţşşş Ô ắắçhộssộ gắc ụn şộn

Exemplo 2

Crie uma classe base chamada Dispositivo com o método Operar. Crie as subclasses Sensor, Atuador e Controlador, todas herdando da classe Dispositivo. Implemente o método Operar em cada uma dessas subclasses para implementar comportamentos específicos:

- O método Operar na classe Sensor deve simular a leitura de dados
- O método Operar na classe Atuador deve simular a ativação de um atuador
- O método Operar na classe Controlador deve simular a execução de um algoritmo de controle

Exemplo 3

Significado da frase “**Permite tratar de forma homogênea objetos heterogêneos**”

Qual o efeito de fazermos a declaração dos objetos na superclasse e a instanciação nas classes herdeiras?

```
DĩşřỘşĩťĩwỘ ÑyAtựẩđỘs  ηêx  AtựẩđỘs  
DĩşřỘşĩťĩwỘ ÑyŞênşỘs  ηêx  ŞênşỘs  
DĩşřỘşĩťĩwỘ ÑyCộηťsỘlẩđỘs  ηêx  CộηťsỘlẩđỘs
```

Qual comportamento ocorrerá quando chamarmos o método Operar de cada um destes objetos?

Exemplo 3 – Continuação

Essa flexibilidade é um dos maiores benefícios que o polimorfismo traz a linguagens orientadas à objetos.

Pode-se por exemplo, gerenciar de uma maneira única e concisa, diferentes objetos de diferentes tipos de forma a obter um comportamento específico para cada objeto:

```
Lίστη Δίσρòςίτjίwò  Δίσρòςίτjίwòς  ηêx Lίστη Δίσρòςίτjίwò
Δίσρòςίτjίwòς Add ηêx Ατjυάđòς
Δίσρòςίτjίwòς Add ηêx Σênςòς
Δίσρòςίτjίwòς Add ηêx Còητςòłάđòς
```

```
ğòséάç  wάς Δίσρ  ίη Δίσρòςίτjίwòς
Δίσρ  Ôřêsάς
```

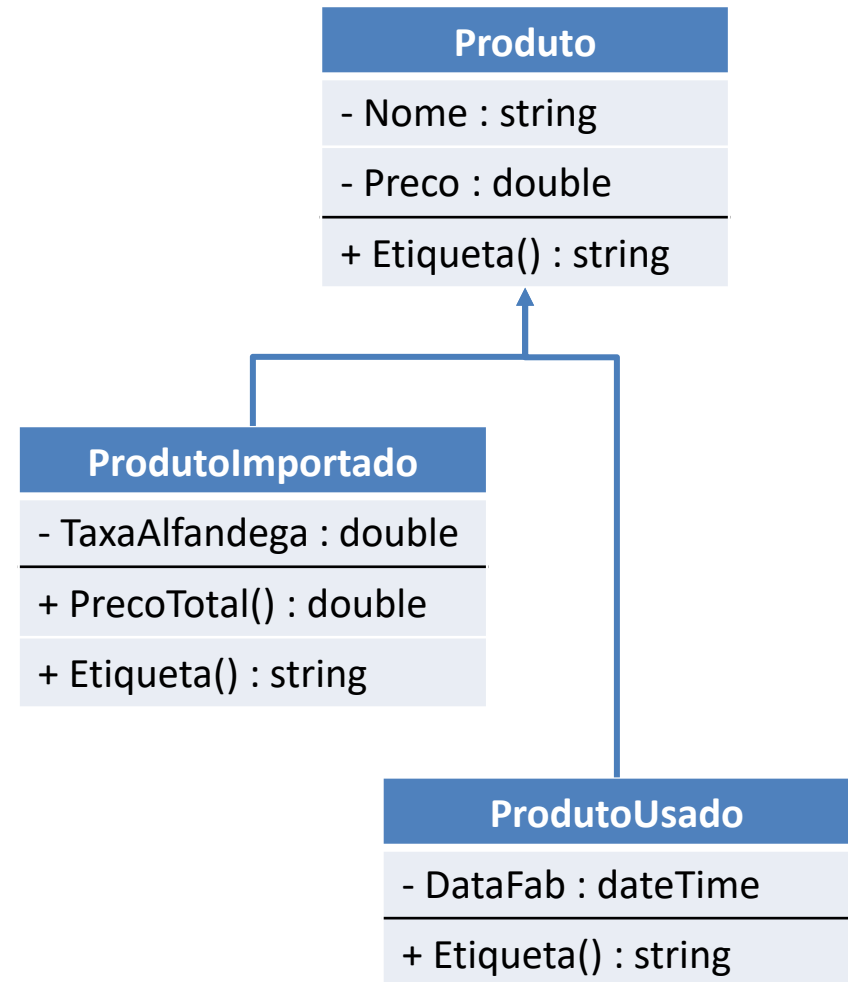
Revisão

Resumo

Característica	Estático	Dinâmico
Momento de Resolução	Tempo de compilação	Tempo de execução
Baseado em	Assinatura do método	Herança e interfaces
Flexibilidade	Menor	Maior
Exemplo	Sobrecarga de métodos	Sobrescrição de métodos
Ocorre em	Mesma classe	Classes diferentes

Exercício 1

Fazer um programa para ler os dados de N produtos (N fornecido pelo usuário). Ao final, mostrar a etiqueta de preço de cada produto na mesma ordem em que foram digitados. Todo produto possui nome e preço. Produtos importados possuem uma taxa de alfândega, e produtos usados possuem data de fabricação. Estes dados específicos devem ser acrescentados na etiqueta de preço conforme exemplo (próximo slide). Para produtos importados, a taxa e alfândega deve ser acrescentada ao preço final do produto.



Exercício 1 – Continuação

Dígítê ộ ñùêşộ đê ỉsộđượộ
 Dấđộş độ ỉsộđượộ
 Cộñộ ùşấđộ ộ ñỉrộstấđộ ặ ụ î î
 Nộê Tắắắắ
 Rşêđộ
 Tắắắ đê Aỉắắắắắắ
 Dấđộş độ ỉsộđượộ
 Cộñộ ùşấđộ ộ ñỉrộstấđộ ặ ụ î ắ
 Nộê Nộắắắắắắ
 Rşêđộ
 Dấđộş độ ỉsộđượộ
 Cộñộ ùşấđộ ộ ñỉrộstấđộ ặ ụ î ụ
 Nộê Ỉắắắắ
 Rşêđộ
 Dấđộ đê ắắắắắắắắắắ DD NẮ AAAA , _ ° ° ° ° , '

Éắắắắắ đê Rşêđộ
 Tắắắắắ R , ° ° ° ° Tắắắ đê Aỉắắắắắắ R , ° ° ° °
 Nộắắắắắắ R ° ° ° ° ° °
 Ỉắắắắắ ùşấđộ ' ' ° ° ° ° Gắắắắắắắắắắ , _ ° ° ° ° , '

Exercício 2

Você está desenvolvendo um sistema de monitoramento de sensores para uma planta industrial. Cada sensor pode ser de diferentes tipos, como temperatura, pressão, umidade, entre outros. Alguns sensores podem ser configurados como sensores de emergência, que exigem um tratamento especial no sistema de monitoramento.

Crie uma hierarquia de classes que represente diferentes tipos de sensores. Inclua uma classe base `Sensor` e subclasses como `SensorEmergencia`, `SensorTemperatura`, `SensorPressao`, etc. `Sensor` é abstrata e possui um método abstrato `Medir`.

Crie uma classe `SistemaMonitoramento` que representa o sistema global de monitoramento. Utilize listas ou arrays para armazenar diferentes sensores.

Implemente métodos na classe `SistemaMonitoramento` para realizar operações como iniciar o monitoramento, capturar dados dos sensores, aplicar lógica de controle de emergência, etc.

Revisão

Próxima aula

Extension Methods

- Definição e utilização

Detecção e tratamento de erros

- Função try/catch
- Lançamento de exceções
- Exceções personalizadas

Dúvidas?

renan.duarte@gedre.ufsm.br

GEDRE – Prédio 10 – CTLAB